



MODUL 9 · DATABASE DENGAN PRISMA 7

Prisma Schema, Model & Relasi



Schema adalah satu sumber kebenaran untuk database, types, dan client — semua sinkron otomatis.

Anatomi prisma/schema.prisma

Tiga bagian: generator, datasource, dan model definitions.

prisma/schema.prisma — Prisma 7

```
// — 1. Generator — konfigurasi Prisma Client —————
generator client {
  provider = "prisma-client" // bukan prisma-client-
jsoutput  = "../generated/prisma" // wajib di Prisma 7
}

// — 2. Datasource — koneksi database —————
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL") // baca dari .env
}
// ⚠ Di Prisma 7, url juga bisa di prisma.config.ts

// — 3. Model — definisi tabel —————
model User {
  id          String   @id @default(cuid())
  email       String   @unique
  name        String
  role        UserRole @default(READER)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  posts       Post[] // relasi one-to-many
}

enum UserRole { READER AUTHOR ADMIN }
```

Field Attributes

@id — primary key

@unique — nilai unik

@default(value) — nilai default

Tipe default:

cuid() — ID pendek, sortable

uuid() — UUID v4 standard

autoincrement() — integer auto

now() — timestamp saat ini

@updatedAt — auto update saat record berubah

@map("column_name") — nama kolom berbeda di DB

@@map("table_name") — nama tabel berbeda di DB

Relasi: One-to-Many, Many-to-Many, One-to-One

Prisma mendefinisikan relasi dengan @relation decorator — mudah dibaca dan dipahami.

```
// — One-to-Many: User memiliki banyak Post —————
model User {
  id    String @id @default(cuid())
  name  String
  posts Post[] // relasi: user → many posts
}
model Post {
  id      String @id @default(cuid())
  title   String
  authorId String // foreign key — wajib ada
  author  User   @relation(fields: [authorId], references: [id], onDelete:
    Cascade)

// — Many-to-Many: Post memiliki banyak Tag (via implicit join table) —————
model Post { tags Tag[] }
model Tag   { posts Post[] }
// Prisma buat join table otomatis: _PostToTag

// — Many-to-Many explicit (dengan data di join table) —————
model Post { tagLinks PostTag[] }
model Tag   { postLinks PostTag[] }
model PostTag {
  postId  String
  tagId    String
  addedAt DateTime @default(now())
  post     Post @relation(fields: [postId], references: [id])
  tag      Tag  @relation(fields: [tagId], references: [id])
  @@id([postId, tagId]) // composite primary key
}

// — One-to-One: User memiliki satu Profile —————
model User { profile Profile? }
model Profile { userId String @unique; user User @relation(...) }
```

prisma.config.ts — File Konfigurasi Baru

Menggantikan prisma block di package.json. TypeScript native, lebih powerful.

prisma.config.ts — taruh di root project

```
import 'dotenv/config'
import { defineConfig, env } from 'prisma/config'

export default defineConfig({
  // Lokasi schema.prisma
  schema: 'prisma/schema.prisma',

  // Konfigurasi migration
  migrations: {
    path: 'prisma/migrations', // folder migrations
    seed: 'tsx prisma/seed.ts', // perintah seed
  },

  // Datasource — URL database
  datasource: {
    url: env('DATABASE_URL'), // type-safe env
    // !env() lebih aman dari process.env:
    // akan error jika variabel tidak ada
  },
})

// ⚠ prisma.config.ts WAJIB untuk:
// - prisma migrate dev
// - prisma migrate deploy
// - prisma db push
// Tanpa file ini, CLI commands akan error di Prisma 7
```

env() vs process.env

env() dari prisma/config:

- Type-safe — error jika tidak ada
- Lebih eksplisit

process.env.DATABASE_URL:

- Tidak type-safe
- Mungkin undefined

Keduanya OK, tapi env() direkomendasikan di prisma.config.ts

Catatan:

prisma.config.ts tidak menggantikan .env!
Tetap butuh .env untuk menyimpan nilai variabel.

prisma.config.ts hanya mendefinisikan bagaimana Prisma CLI membaca konfigurasi.

Evaluasi Pemahaman

Jawab sebelum lanjut.

Q1 — Di Prisma 7, apa yang WAJIB ditambahkan di generator block selain provider?

A) engineType = "client"

B) output = "../path/to/generated" (lokasi file yang di-generate)

C) target = "node"

Q2 — Apa perbedaan Many-to-Many implicit dan explicit di Prisma?

A) Tidak ada bedanya

B) Explicit pakai model join table sendiri — berguna jika join table butuh field tambahan

C) Explicit hanya untuk PostgreSQL

Q3 — prisma.config.ts dipakai oleh apa?

A) Runtime aplikasi Next.js

B) Prisma CLI (migrate, push, dll) — bukan runtime aplikasi

C) Kedua-duanya

Tugas Mandiri

Definisikan schema lengkap untuk project blog.

01 `schema.prisma`

Buat schema dengan model: User (id, email, name, role, createdAt, updatedAt), Post (id, title, slug, content, published, viewCount, authorId), Comment (id, content, postId, authorId).

02 Relasi

Definisikan relasi: User→Post (one-to-many), Post→Comment (one-to-many), User→Comment (one-to-many). Tambahkan onDelete: Cascade ke semua FK.

03 `prisma.config.ts`

Buat prisma.config.ts di root project. Set schema path, migration path, dan seed command. Test dengan bunx prisma format (format schema).

04 Validasi Schema

Jalankan bunx prisma validate. Fix semua error. Jalankan bunx prisma format untuk format otomatis. Commit schema yang bersih.



bunx prisma format otomatis format schema.prisma. Jalankan setelah setiap perubahan schema.

Yang Sudah Kita Pelajari

- ✓ Schema.prisma: generator (output wajib!) + datasource + models.
- ✓ Field attributes: @id, @unique, @default(cuid()), @updatedAt, @relation.
- ✓ Relasi: one-to-many, many-to-many implicit, many-to-many explicit (pakai join model).
- ✓ prisma.config.ts: konfigurasi CLI. Wajib untuk migrate commands di Prisma 7.
- ✓ env() dari prisma/config lebih aman dari process.env untuk prisma.config.ts.

→ Selanjutnya: **Bab 4 — Driver Adapter & Singleton di Next.js**

